

11.1 Generative Adversarial Nets

A generator fakes samples. A discriminator tries to tell them from the data.

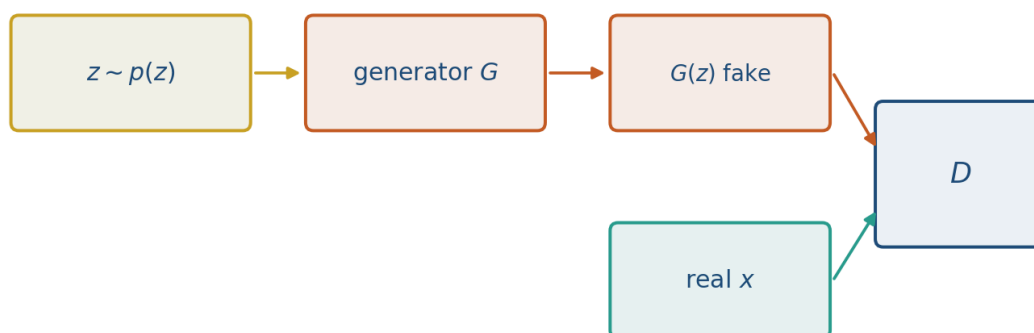
These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

A **GAN** (Goodfellow et al., 2014) is two networks in a contest: a generator that paints fakes, and a discriminator that tries to catch them. You do not write down $p(x)$ and maximize likelihood. You train a sampler.

1. The generator

Noise z (usually isotropic Gaussian) in; a sample $x = G(z; \theta_g)$ out, same shape as a real observation. G is a decoder without an encoder: it does not reconstruct a particular training point. It maps latent draws into data space.

GAN: G tries to fool D ; D tries not to be fooled



The job of G is to make the **distribution** of fakes look like the distribution of reals. One pretty sample is not a trained generator. If every z maps to the same pretty point, G has failed, even if that point fools a human.

In this course G is a tiny MLP (Lab 11: 8-D noise $\rightarrow$ 2-D point). The same picture is a CNN or a transformer decoder when x is an image or a spectrogram. The algebra does not change: sample z , push it through G , hand the fake to D .

2. The discriminator

$D(x; \theta_d)$ is a binary classifier: the probability that x is real. On a real batch it should sit near 1; on $G(z)$ it should sit near 0. It never sees z . It only sees the sample.

$$y = D(x; \theta_d) \in (0, 1).$$

Then $1 - D(x)$ is the probability it calls the point fake.

D is ordinary supervised learning on a moving dataset: reals with label 1, fakes with label 0. The catch is that the fake distribution changes every time you update G . You never have a fixed classification problem.

3. Why two players

If you trained only G against a frozen D , G would overfit whatever D currently likes. If you trained only D , it would become a perfect detector of yesterday's fakes and give G a useless (flat) gradient. The pair has to stay in tension. Note **11.2** writes that as a min-max game; note **11.3** is what happens when G finds one easy fake and stops covering the rest.

For this course, GANs are the first **implicit** generator you meet. Diffusion (Week 12) will give you an explicit noise schedule instead of an adversary. Text-to-image later in the module still needs a way to **condition** (Week 5's CLIP is one).

Draw the figure on the board and leave it up: $z \rightarrow G \rightarrow \text{fake}$ into D , real x into D . Everything later in Week 11 is a comment on that picture.

Name what you cannot do: $p(x)$, $p(z | x)$, a test log-likelihood. Name what you can do: draw fakes in one forward pass. That pair of sentences is the exam answer for "why a GAN vs why a diffusion model."

4. Teaching this note

About **35 minutes** at the board, then **~8 minutes** of video. This is the first of three GAN notes in a two-hour class; Lab 11 uses the rest of the block.

- **0–12 min.** Implicit vs explicit. You cannot write $p(x)$ for a GAN. You can sample. Sketch the two-player diagram.
 - **12–24 min.** What G and D each see. G never sees a real x on its own update. D never sees z .
 - **24–34 min.** Worked 1-D two-mode example. Leave the numbers on the board; note **11.3** will reuse them.
 - **Then** play CS231N **46:45–54:00** (GAN setup: give up on density, noise through a generator). Pause when the two-player cartoon appears.
-

5. Worked example

Reals are two spikes in 1-D: half the mass at $x = -2$, half at $x = +2$. (Lab 11 is the same idea in 2-D, means $(-2, 0)$ and $(2, 0)$.)

Suppose an untrained G ignores z and always emits $G(z) = +2$.

A Bayes-optimal D then scores:

location	what D sees	$D(x)$
$x = +2$	half real, half fake	0.5
$x = -2$	only reals	1
anywhere else	only fakes, if any	near 0

G is “winning” at $+2$ in the sense that $D(G(z)) = 0.5$, a coin flip. It is losing as a **distribution**: the left mode has zero fake mass. One sample at $+2$ looks perfect. The generator is not trained.

If you now let G move and it jumps all mass to -2 , the table flips: $D(-2) = 0.5$, $D(+2) = 1$. That oscillation is the seed of note **11.3**.

6. Where students get stuck

- Treating one pretty fake as success. G is a distribution, not a Photoshop filter.
- Thinking G is trained by comparing $G(z)$ to the nearest real (an autoencoder). It is not. The only teacher is D 's scalar.
- Forgetting that D 's data distribution moves. A D that was perfect last step is scoring a different G this step.

7. Video

Watch [Stanford CS231N 2017 lecture 13, Generative Models](#), **46:45–54:00**.

Pause at the two-player setup: noise z through a generator, no explicit density. Skip PixelCNN/VAE (the first 45 minutes) in class; those are not this note. Notes **11.2** and **11.3** reuse the same URL at later pause points.

8. Practice

1. G never sees a real image on its own update (only through D 's scores). How can it learn the data distribution?
2. Why is a perfect D (always 1 on reals, always 0 on fakes) a problem for G 's gradient?
3. Name one task where a GAN is a natural fit and one where an explicit likelihood model would be easier to evaluate.
4. In the 1-D example, $G(z) = +2$ always and reals are equal spikes at ± 2 . What is the Bayes-optimal $D(+2)$? What is $D(-2)$?
5. You draw 10 fakes and all 10 equal $+2$. A classmate says “ D is 0.5 there, so G is done.” Write one sentence that disagrees, using coverage.